Topic of the paper (project):

Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks

Authors: **Jun-Yan Zhu, Taesung Park, Phillip Isola, Alexei A. Efros**

Link to the paper page: https://arxiv.org/abs/1703.10593

Alireza Alimirzaei – 40011141054446

Final computer vision Project

Course: Fundamentals of Computer Vision

Instructor: Professor Soltani

Note: After more extensive research into the methods used for image-to-image translation in the original paper, I decided to use the updated version of this paper. Instead of conditional adversarial networks (cGAN), which require a paired image to be given as input, it uses cycle-consistent adversarial networks (Cycle-GAN), which work without the need for paired data and are much more practical for image-to-image translation. I hope you will forgive this difference in method compared to the original paper; honestly, the second method was more appealing to me because it has more up-to-date applications. Thank you.

Summary Report of the Paper:

Introduction:

Image-to-image translation is one of the classic problems in machine vision and image processing, in which the goal is to convert an image from one domain (such as horse) to another domain (such as zebra). In older methods, and even in deep-learning-based models such as **pix2pix**, this process required **paired training data**, meaning that to train the model, for every image in the source domain there had to be exactly the corresponding image in the target domain. However, in the real world, preparing such paired data is usually very difficult, very time-consuming, and costly. For example, we cannot have a photo of a landscape with a zebra in it in which, at exactly the same moment and with the same background characteristics, etc., a horse is also standing. The main goal of this paper is to provide a framework that can perform this translation using **unpaired data**; that is, a model that, given only a set of images from the input domain X and a completely independent set of images from the output domain Y, learns the mapping between them.

Next, the researchers compared their method with previous methods:

- Unlike pix2pix, which required paired images, this paper turns to unsupervised learning.

- Neural Style Transfer: Methods such as the work of Gatys et al. transfer the style of one image to another image, but their limitation is that they work on a single image, whereas CycleGAN, instead of learning the style of one specific image, learns the mapping between entire collections of images.

- Cycle Consistency: The idea of using cycles had previously been used in natural language translation and in motion tracking in videos, but this paper applies it in a novel way to the training of unpaired GANs.

Formulas and mathematics used in Cycle-GAN:

Suppose we have two input image domains, X and Y. The goal is to learn the following two mapping functions (generators):

Example: function G for converting a horse to a zebra

$$G : X \rightarrow Y$$

Example: function F for converting a zebra to a horse

$$F : Y \rightarrow X$$

To evaluate the quality of the generated images, two discriminators are also introduced:

Discriminator Dy tries to distinguish between real images of domain Y and images generated by function G.

Discriminator Dx tries to distinguish between real images of domain X and images generated by function F.

Loss functions:

- Adversarial Loss

It is defined for function G and discriminator Dy as follows:

$$\begin{aligned}\mathcal{L}_{\text{GAN}}(G, D_Y, X, Y) = &\mathbb{E}_{y \sim p_{\text{data}}(y)}[\log D_Y(y)] \\ &+ \mathbb{E}_{x \sim p_{\text{data}}(x)}[\log(1 - D_Y(G(x))],\end{aligned}$$

The discriminator tries to maximize this value (accurately distinguishing real from fake).

Generator G tries to minimize this value (fooling the discriminator).

The same formula is used symmetrically for generator F and discriminator Dx as well.

In fact, if the discriminator makes a mistake, then according to the formula it receives a heavy penalty of negative infinity, which I will explain further in the implementation section.

- Cycle Consistency Loss

Using only the adversarial loss is not sufficient for unpaired data, because the model suffers from **mode collapse** (for example, it maps all inputs to one fixed output image). To solve this problem, the cycle consistency rule is applied. This rule says that if we take an image from domain X to Y and then bring it back to X, we should arrive at the original image, otherwise a penalty is received. This loss is formulated using the L1 norm as follows:

$$\begin{aligned}\mathcal{L}_{\text{cyc}}(G, F) = &\mathbb{E}_{x \sim p_{\text{data}}(x)}[\|F(G(x)) - x\|_1] \\ &+ \mathbb{E}_{y \sim p_{\text{data}}(y)}[\|G(F(y)) - y\|_1].\end{aligned}$$

Normally, with a single generator for converting a horse to a zebra, the generator can gradually learn to produce an image that looks very much like a real zebra, so that it fools the discriminator every time: for every input image it receives, it hands the discriminator the same flawless zebra image it generated earlier and fools it. This is technically called mode collapse. To prevent this, two generators are used here, so that the zebra image generated by the first generator is converted back into a horse, and, by evaluating its difference from the original input images, the components (generators and discriminators) are penalized according to the amount of error in each part.

- Full Objective

Finally, the overall objective of the model is a combination of the above loss functions, which, as you will see in the implementation section, is divided between the generators and discriminators according to each one's share of the error, and in which the coefficient λ specifies the weight of the cycle loss; in the paper, the value of lambda is assumed to be 10.

$$\begin{aligned}\mathcal{L}(G, F, D_X, D_Y) =& \mathcal{L}_{\text{GAN}}(G, D_Y, X, Y) \\ &+ \mathcal{L}_{\text{GAN}}(F, D_X, Y, X) \\ &+ \lambda \mathcal{L}_{\text{cyc}}(G, F),\end{aligned}$$

Finally, the goal is to find the appropriate weights for this minimax equation:

$$G^*, F^* = \arg\min_{G,F} \max_{D_x, D_Y} \mathcal{L}(G, F, D_X, D_Y).$$

- Identity Loss

In some tasks, such as converting paintings to photographs, the identity loss is used to preserve the color palette of the input image. This function says that if we directly feed generator G, whose job is to convert X to Y, an image from domain Y, it should not change it, otherwise the generator is penalized:

$$\mathcal{L}_{\text{identity}}(G, F) = \mathbb{E}_{y \sim p_{\text{data}}(y)}[\|G(y) - y\|_1] + \mathbb{E}_{x \sim p_{\text{data}}(x)}[\|F(x) - x\|_1].$$

Generator and discriminator network architecture:

- Generator network: It is designed based on the architecture of Johnson et al. This network uses a number of ResNet neural network blocks: for images with a resolution of 128×128 it uses 6 ResNet blocks, and for images of 256×256 and higher it uses 9 ResNet blocks. This architecture allows the network to preserve the spatial information of the source image with high accuracy. (More details will be given in the simulation section.)

- Discriminator network: The **PatchGAN** architecture with 70×70 dimensions is used. Instead of evaluating the whole image, PatchGAN divides the image into smaller parts and checks whether each part is real or fake separately, and finally the sum of the real/fake values of all the parts is computed. This makes the model focus more on details and textures

and is also computationally lighter, whereas previously the discriminator assigned a single random or estimated value to the entire image, which lowered the model's accuracy.

Training stability techniques used:

- Use of LSGAN: Instead of using the logarithm in the adversarial loss function, they used mean squared error, which makes the training process much more stable.

- Use of image history: To prevent oscillation of the discriminators, the discriminator is trained with a pool (Buffer) containing 50 previously generated images, not only the images generated in that same step (Iteration).

- The optimization algorithm used is **Adam** with a learning rate of 0.0002 for the first 100 epochs (Epoch), and in the second 100 epochs the learning rate decreases linearly to zero, so that early in training the model has a freer hand to change the weights, and after a relative amount of learning the learning rate is gradually reduced so that accuracy increases.

The paper then evaluates the model's performance, applications, and limitations, which are summarized below:

- Machine and human evaluation: The model's performance has been demonstrated using computer object recognition capability (FCN) and the degree to which it fools the human eye (AMT).

- Impressive competitive performance: The model was able to beat all previous unpaired methods and come very close to the quality of paired models such as pix2pix.

- Artistic style transfer: This network successfully converts real photographs into the painting style of famous artists such as Van Gogh and Monet.

- Object transfiguration: The ability to change the appearance of animals or plants (such as turning a horse into a zebra or an apple into an orange) is one of its prominent applications.

- Season transfer: This model can realistically change the weather and seasonal conditions of a landscape (for example, from winter to summer).

- Photo enhancement and realism: It is possible to generate natural photos from paintings and to create a background-blur effect similar to that of professional cameras.

- The challenge of geometric changes: CycleGAN is excellent at changing color and texture, but performs poorly on severe structural and geometric changes (such as turning a dog into a cat).

- Dataset bias errors: Due to the limitations of the training dataset, the model sometimes makes strange mistakes (for example, striping a person riding a horse while generating a zebra).

- Quality gap with supervised models: Despite many advances, in some scenarios the model's results still do not match those of methods that require paired data.

Conclusion: This architecture is a major leap in image processing, but it is still not flawless in understanding complex geometric changes and in overcoming the limitations of the input data.

Summary Report of the Personal Simulation:

Introduction:

In this implementation, the method presented in the paper was re-implemented independently: a nine-block ResNet generator, a PatchGAN discriminator, and an LSGAN objective, together with the cycle-consistency constraint and the identity constraint, are used exactly according to the architecture and training details reported in the paper. The code was written from scratch and independently of the authors' official GitHub code, both as an implementation exercise and so that its results could be compared with the paper. In addition to reproducing the baseline model, a three-part ablation study was designed that examines the effect of the identity constraint and of the weight of the cycle constraint on the quality of the generated images.

- The code is written in **Python** with the **PyTorch** library (the main deep learning framework), plus **torchvision**, which was used for image processing and dataset loading,

- The dataset used is **horse2zebra** (1067 horse images and 1334 zebra images) with a resolution of 256×256.

- The code was trained on a personal computer with a 1070 Ti graphics card, so the number of epochs was reduced to 30 for each of the 3 training runs (due to time constraints). Even so, a total of 30 hours was spent on training, so I apologize for not training each configuration for 200 epochs, but all the criteria, such as the weight reduction after half of the epochs have elapsed, etc., have been observed.

- The code is written modularly in 6 files: models.py, datasets.py, utils.py, train.py, test.py, metrics.py.

Neural network architecture (models.py)

This module contains the basic architectures of the generator and discriminator networks; it is designed so that it can process images at different resolutions.

- The generator network design is based on **ResNet** blocks and consists of three main parts:

Encoder: Consists of 3 convolutional layers that extract features from the input image and reduce its spatial dimensions using a stride of 2 (Downsampling); this reduces the computational load in the subsequent layers.

Transformer: The core of the network, which consists of 6 ResNet blocks for 128×128 images or 9 ResNet blocks for images of 256×256 and above. Each ResNet block consists of two 3×3 convolutional layers together with InstanceNorm2d normalization layers. The skip

connections in these blocks ensure that the structural features of the image are not lost during the transformation process and that the vanishing gradient problem does not occur.

Decoder: Consists of transposed convolution layers (Fractional-strided Convolutions) to restore the feature map to its original dimensions, i.e. (Upsampling). The output of this part passes through a Tanh activation function so that the pixel values lie in the range [-1, 1].

- Discriminator network (Discriminator)

The discriminator is implemented using the **PatchGAN** architecture.

- Instead of outputting a single number (zero or one) for the whole image, this network outputs a 2D matrix (for example 30×30), where each entry corresponds to a part of the original image (a patch (Patch)) of 70×70 pixels.

- This architecture uses convolutional layers with the LeakyReLU(0.2) activation function and InstanceNorm2d normalization.

- The advantage of this method is its precise focus on textures and local details, as well as a drastic reduction in the number of network parameters.

Data preparation and distributed management (datasets.py)

Data management in the unpaired scenario (Unpaired) is one of the main challenges of the simulation.

- The UnpairedDataset class: This custom class, which inherits from torch.utils.data.Dataset, receives the paths of two separate image folders (for example, domain A and domain B).

- Dataset size mismatch: Since the number of images in the two domains is not necessarily equal, the algorithm is designed to use the modulo operator (Modulo) to repeat the smaller domain, so that in each epoch all the images of the larger domain are processed at least once.

- Preprocessing (Data Augmentation): Using the torchvision.transforms library, a preprocessing pipeline is applied:

- Applying an initial enlargement of the image to a slightly larger resolution, e.g. 286×286.

- Applying a random crop (RandomCrop) to the final network input size (256×256).

- Applying a random horizontal flip with a probability of 50 percent (RandomHorizontalFlip).

- Normalizing the tensors to standard values with a mean of 0.5 and a standard deviation of 0.5 for the three color channels (RGB); to increase the model's discrimination ability, we convert the light intensity of colors that lie in the middle to a value of 1 or negative 1.

Stability mechanism during training (utils.py)

Training four networks simultaneously (two generators and two discriminators) has a high potential for instability, therefore:

- Image pool (ImagePool): To prevent severe oscillations of the discriminators, a class called ImagePool was implemented, which keeps a cache with a capacity of 50 recently generated images. At each discriminator training step, the system uses the image generated at that moment with 50 percent probability, and with 50 percent probability gives the discriminator a random image from the ImagePool history. This critical technique prevents the discriminator from forgetting previous patterns.

- Weight Initialization: The weights of all convolutional layers are randomly initialized from a normal distribution with a mean of 0 and a standard deviation of 0.02, to ensure a strong and unbiased start for the networks.

Training loop and loss functions (train.py)

This module is responsible for coordinating the computational graph and the optimization.

- Least squares loss (MSELoss): To resolve the vanishing gradient problem in the standard loss function of adversarial networks, the LSGAN idea was used and mean squared error was substituted for it.

- Cycle consistency loss (L1Loss): If an image goes from A to B and then returns to A, the difference between the final output and the original image is measured by the Manhattan distance (L1) metric. This loss is combined into the total objective function with coefficient λ = 10.

- Optimizer (Adam Optimizer): Two separate optimizers are defined: one for the combination of generators G and F, and the other for the combination of discriminators Dx and Dy, with a base learning rate of 0.0002 and momentum coefficient β_1 = 0.5.

- Learning rate scheduling (LR Scheduler): Training is set for 200 epochs. In the first 100 epochs the learning rate remains constant, and in the second 100 epochs, using LambdaLR, it decreases linearly and reaches zero, so that the model stabilizes at the global minimum (Global Minimum).

Inference phase and qualitative generation (test.py)

After the final weights are saved, this module is used for qualitative validation.

- The generator networks are put into eval mode so that layers such as InstanceNorm have fixed, deterministic behavior.

- The operation is run under the torch.no_grad context; by turning off gradient computation, GPU memory consumption is drastically reduced and inference speed is maximized.

- The output images are converted from the range [-1, 1] back to the range [0, 255] and saved as .png or .jpg files in the results folder.

Quantitative evaluation and metrics (metrics.py)

So that the evaluation is not limited to visual observation, the system is equipped with a quantitative evaluator:

- FID metric (Frechet Inception Distance): In this file, a powerful pretrained feature-extraction model, the InceptionV3 network, is used.

- The real images of the target domain and the images generated by the generator are fed into this network, and their deep feature vectors are extracted.

- Then the mean (μ) and the covariance matrix (Σ) of these features are computed for both sets, and the mathematical distance between these two distributions is obtained with the following formula:

$$FID = ||\mu_{real} - \mu_{fake}||^2 + Tr(\Sigma_{real} + \Sigma_{fake} - 2(\Sigma_{real}\Sigma_{fake})^{1/2})$$

A low FID score in the output of this module is evidence of the success of the simulation and of the high realism of the generated images.

Review and comparison of the personal implementation results with the paper's results:

Note: As I stated before, due to a lack of time and resources, all the results are compared using 30 epochs of training, and because all the rules of the paper were observed with high precision in the implementation, I have regarded the default implemented model as the trained model of the paper's authors (the truth is that I did not have time to train the authors' model for 30 epochs, but I guarantee with high confidence that the results would be similar).

Here, three training modes have been considered for comparison:

- Base model: In the base model, everything stated above is observed.

- no_Identity model: In this model the identity loss is removed (if we directly feed generator G, whose job is to convert X to Y, an image from domain Y, it should not change it, otherwise the generator is penalized). This means the model may change the background color, lighting, or overall tone of the photo for no reason, because it suffers no loss for doing so.

- Low_cyc model: In this model the lambda in the Cycle-Consistency loss is reduced from 10 to 2. As a result, the influence of the Cycle-Consistency loss is greatly reduced compared to the normal mode, so the model no longer receives enough pressure to preserve the main structure (body shape, pose, details), and the output may therefore become structurally unrelated or distorted.

Quantitative comparison of the results (FID): the lower the FID, the better our model is.

```
FID(./datasets/horse2zebra/testB , ./results/horse2zebra115/fakeB) = 92.305
```
Normal mode: 92.305

```
FID(./datasets/horse2zebra/testB , ./results/no_identity/fakeB) = 119.847
```
no_Identity mode: 119.847

```
FID(./datasets/horse2zebra/testB , ./results/low_cyc/fakeB) = 76.700
```
Low_cyc mode: 76.700

Qualitative comparison of the results:



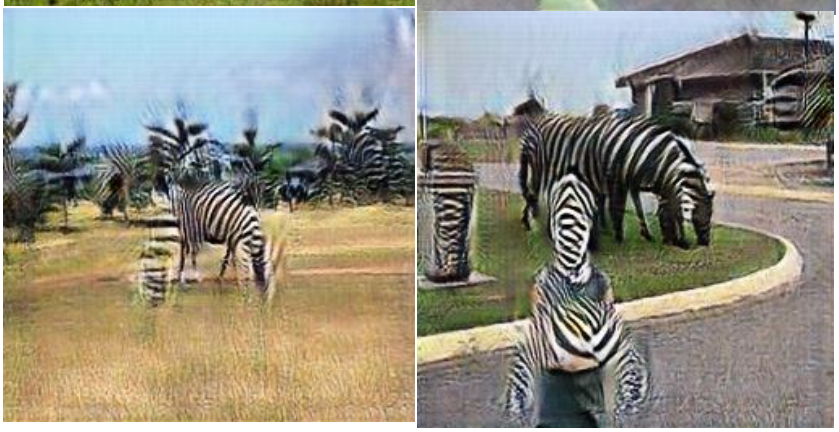
<- Input image



<- Base model output



<- no_identity model output



<- low_cyc model output

n02381460_2050 Horse in a thicket

n02381460_6290 Horses and a little boy

Note: FID only measures how closely, *overall*, the statistical distribution of the generated images resembles the statistical distribution of real zebra photos, and it does not check at all whether that specific horse was actually converted correctly into a zebra (that is, whether the image structure was preserved or not).

The way FID works explains why the low_cyc model gets a better score than the no_Identity model and even than the base model.

Explanation of the comparison results (the image of the little boy and the horses):

- Base model output: As you can see, in the base model output the color of the curb has been preserved fairly well, and the sky and the other elements of the image have also remained largely untouched. However, due to the incomplete training of the model, the horses have not been completely converted into zebras, and the model also mistakenly turns the little boy into a zebra; this problem will be largely resolved with more training and further weight updates.

- no_Identity model output: As you can see, because of the absence of the identity loss and of any penalty on the generator, the model has changed the background colors, the color of the sky, the color of the street curb, the structure and color of the house behind the little boy, the structure of the horses and of the little boy, etc., and the generated image has very low similarity to the original image.

- Low_cyc model output: As you can see, because the influence of the cycle-consistency loss was reduced, there was less pressure on the generators, which led to image details and the boy's body shape not being preserved and the trash can turning into a zebra, due to the reduced pressure to reproduce the original image from the generated image.

Everything stated above also applies to the image of the horse in the thicket.

Conclusion:

The results of these ablation experiments fully confirm the role of each component of the paper's objective function. Removing the identity loss in the no_Identity model caused unwanted changes in the color and texture of the image background, exactly the phenomenon that the authors of the paper mention in section 5.2 and cite as the motivation for adding this loss. Similarly, reducing the weight of the cycle-consistency loss in the Low_cyc model caused the loss of structural details of the image (such as the unwanted conversion of the trash can into a zebra), which shows that this loss plays an effective role in preserving the structure between the source and target images.

However, an unexpected result was observed in the quantitative comparison of the models: the FID value for the Low_cyc model, which performed worst qualitatively, was lower than that of the base model. This inconsistency shows that FID, as a metric that only measures the overall statistical similarity of the distribution of the generated images to that of real images, is unable to detect whether the structural similarity between the input and output images is preserved. This observation in fact clarifies one of the reasons why the authors of the original paper did not use FID and instead relied on human qualitative comparison.

Overall, despite the limitation of training to 30 epochs (instead of the 200 epochs in the paper), which reduced the absolute quality of the results, the relative trend observed among the three configurations is fully aligned with the paper's theoretical claims and confirms the validity of the Cycle-GAN design.